

10. (a) In the State pattern, the current state of the context will change as the program runs, and the behaviour will change for each different state. In the Strategy pattern, the behaviour of the program should not change for each strategy, the result will just be computed using a different algorithm.
- (b) i. If an **Academic** needs to change from a **Lecturer** to a **Professor**, then a new object has to be created, have all the fields of the previous object copied over, and the old object deleted.
- ii. When the rank changes, the current **mRank** object needs to be deleted, but if a reference to it has been given out, then even though it is not in use there may be a dangling reference, and so the object will not be deleted.
- iii. The open-closed principle can allow the creation of immutable objects, as the object being "closed" means it should not be able to be modified, and this can be done by providing no methods that change state in the interface. This could still cause the issue with dangling references.
- (c) i. **draw()** should be an abstract method as every **Shape** subclass should be required to implement it, as it does not make sense to have a shape that cannot be drawn.
- (d) Create two new interfaces, **Wrapped** and **Unwrapped**. Create components using generics, designed to be used with **Wrapped** and **Unwrapped**. When writing a decorator, write it so that it takes in a **Unwrapped** and returns a **Wrapped**.

